

Gestión y Tecnologías de Procesos de Negocio

Práctica: Implementación básica.

Objetivos

El objetivo de esta práctica es introducir al estudiante en las actividades básicas de implementación y ejecución de Procesos de Negocio. Para ello, se utilizará el estándar BPMN 2.0 [1] para el modelado de los procesos y Flowable [2] como plataforma para su modelado, implementación y ejecución. En particular, se utilizará Flowable Modeler para la construcción de los modelos BPMN y el motor de procesos de Flowable 6.8 para su despliegue y ejecución.

Para la instalación y ejecución de Flowable mediante Docker, consultar el Anexo A.

Ejercicios

- a) Utilizando Flowable, cree los usuarios *Alumno1* y *Alumno2*. Luego, cree el grupo (rol) *Alumno* e incluya a ambos usuarios como miembros del grupo.
- b) Mediante ejemplos abstractos, implemente los siguientes patrones¹:
 - 1) Implemente el patrón *Direct Distribution*.
 - 2) Implemente los patrones *Parallel Split* y *Synchronization*.
 - 3) Implemente los patrones *Exclusive Choice* y *Simple Merge*.
 - 4) Implemente los patrones *Multi-Choice* y *Structured Synchronizing Merge*.
 - 5) Implemente el patrón *Multi-Merge*.
 - 6) Implemente el patrón *Arbitrary Cycles*.
 - 7) Implemente el patrón *Multiple Instances without Synchronization*.
 - 8) Implemente el patrón *Multiple Instances with a Priori Design-Time Knowledge*. Considere tanto la ejecución secuencial como la ejecución en paralelo de las múltiples instancias.
 - 9) Implemente el patrón *Deferred Choice*.
 - 10) Experimente con eventos de borde (*Boundary Events*) de tipo *Timer* asociados a actividades, considerando las variantes interruptiva y no interruptiva.
 - 11) Experimente con eventos de cancelación (*Cancel Events*) en el contexto de una transacción BPMN, analizando su efecto sobre la ejecución del proceso. “
 - 12) Implemente el patrón *Role-Based Distribution*.
 - 13) Implemente el patrón *Deferred Distribution*.
 - 14) Implemente el patrón *Multiple Instances with a Priori Run-Time Knowledge*. Utilice una *Script Task* con un script *Groovy* para crear, durante la ejecución del proceso, una colección de elementos. Utilice dicha colección para determinar las instancias de una actividad Multi-Instance. Experimente tanto con ejecución secuencial como paralela.
 - 15) Sobre el modelo anterior, experimente con la propiedad *Completion Condition* de una actividad Multi-Instance. Defina condiciones basadas en la cantidad de instancias completadas y analice qué ocurre con las instancias restantes cuando se satisface la condición.
- c) Diseñe un modelo que incluya dos *pools* y experimente con la comunicación entre ellos mediante eventos de tipo *Signal*.

¹ Para los patrones basados en recursos, utilice al menos dos *User Tasks*, asignadas a alguno de los usuarios y/o al grupo creados anteriormente.

-
- d) Utilizando un formulario, experimente con compuertas de decisión cuyas condiciones dependan de los valores ingresados o seleccionados en dicho formulario.
- e) Implemente y ejecute el proceso de negocio desarrollado en la práctica anterior, *Pedido de Pizzas*. Considere los siguientes aspectos:
- Simule la recepción de los pedidos mediante formularios.
 - Simule el envío de las pizzas, la recepción de quejas y el envío del pago mediante eventos de tipo *Signal*.
 - Cree los usuarios y grupos necesarios y asigne las *User Tasks* correspondientes aplicando alguno de los patrones de distribución de recursos estudiados.

A Ejecución de Flowable mediante Docker

La siguiente configuración permite ejecutar Flowable mediante Docker utilizando H2 como base de datos en memoria. Esta alternativa resulta adecuada para realizar pruebas y actividades prácticas, ya que no requiere la instalación ni configuración de un sistema gestor de bases de datos externo.

Para ejecutar Flowable 6.8 utilizando Docker, descargar en primer lugar la imagen correspondiente:

```
docker pull flowable/flowable-ui:6.8.0
```

Luego, crear y ejecutar el contenedor:

```
docker run -d --name flowable \
  -p 8080:8080 \
  flowable/flowable-ui:6.8.0
```

Una vez iniciado el servidor, acceder mediante un navegador web a:

```
http://localhost:8080/flowable-ui
```

Las credenciales predeterminadas son:

```
Usuario: admin
Clave:   test
```

Para detener el contenedor:

```
docker stop flowable
```

Para iniciarlo nuevamente:

```
docker start flowable
```

Para verificar el estado del contenedor:

```
docker ps
```

En caso de ser necesario, los mensajes generados durante la ejecución pueden consultarse mediante:

```
docker logs -f flowable
```

B Ejecución de Flowable con PostgreSQL y pgAdmin 4

Para disponer de una instalación persistente de Flowable se puede utilizar PostgreSQL como sistema gestor de base de datos. De esta forma, los datos generados por Flowable, tales como usuarios, grupos, definiciones de procesos, instancias, tareas y variables, permanecen almacenados aun cuando los contenedores sean detenidos o recreados.

Adicionalmente, puede utilizarse pgAdmin 4 como herramienta gráfica para la administración y consulta de la base de datos PostgreSQL.

Configuración mediante Docker Compose

Crear un directorio para la instalación y, dentro del mismo, los archivos `docker-compose.yml` y `.env`.

El archivo `.env` contiene la configuración de la base de datos:

```
POSTGRES_DB=flowable
POSTGRES_USER=flowable
POSTGRES_PASSWORD=flow10
```

La contraseña utilizada en este ejemplo tiene únicamente fines didácticos. En una instalación de producción debe utilizarse una contraseña segura.

El archivo `docker-compose.yml` puede definirse de la siguiente manera:

Si se utiliza una versión reciente de Docker Compose, esta línea puede omitirse.

```
version: '3.7'
```

```
services:
```

```
  postgres:
```

```
    image: postgres:15
```

```
    container_name: flowable-postgres
```

```
    restart: unless-stopped
```

```
    environment:
```

```
      POSTGRES_DB: ${POSTGRES_DB}
```

```
      POSTGRES_USER: ${POSTGRES_USER}
```

```
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
```

```
    volumes:
```

```
      - postgres_data:/var/lib/postgresql/data
```

```
    healthcheck:
```

```
      test:
```

```
        ["CMD-SHELL",
```

```
          "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
```

```
        interval: 10s
```

```
        timeout: 5s
```

```
        retries: 10
```

```
    networks:
```

```
      - flowable-net
```

```
  flowable:
```

```
    image: flowable/flowable-ui:6.8.0
```

```
    container_name: flowable-persistente
```

```
    restart: unless-stopped
```

```
    depends_on:
```

```
      - postgres
```

```
    environment:
```

```
      SPRING_DATASOURCE_DRIVER_CLASS_NAME:
```

```
        org.postgresql.Driver
```

```
      SPRING_DATASOURCE_URL:
```

```
        jdbc:postgresql://postgres:5432/${POSTGRES_DB}
```

```
      SPRING_DATASOURCE_USERNAME: ${POSTGRES_USER}
```

```
      SPRING_DATASOURCE_PASSWORD: ${POSTGRES_PASSWORD}
```

```
    ports:
```

```
      - "127.0.0.1:8080:8080"
```

```
    networks:
```

```
      - flowable-net
```

```
  pgadmin:
```

```
    image: dpage/pgadmin4
```

```
    container_name: flowable-pgadmin
```

```
    restart: unless-stopped
```

```
environment:
  PGADMIN_DEFAULT_EMAIL: admin@localhost.com
  PGADMIN_DEFAULT_PASSWORD: admin123
ports:
  - "127.0.0.1:5050:80"
volumes:
  - pgadmin_data:/var/lib/pgadmin
networks:
  - flowable-net
```

```
volumes:
  postgres_data:
  pgadmin_data:
```

```
networks:
  flowable-net:
    driver: bridge
```

Antes de iniciar los contenedores puede verificarse la configuración mediante:

```
docker-compose config
```

Para crear e iniciar los contenedores ejecutar:

```
docker-compose up -d
```

El estado de los contenedores puede verificarse mediante:

```
docker ps
```

Deberán encontrarse en ejecución los contenedores correspondientes a Flowable, PostgreSQL y pgAdmin 4.

Acceso a Flowable

Una vez iniciado el entorno, Flowable puede utilizarse desde un navegador web accediendo a:

<http://localhost:8080/flowable-ui>

Acceso a pgAdmin 4

La interfaz web de pgAdmin 4 se encuentra disponible en:

<http://localhost:5050>

Con la configuración anterior, las credenciales de acceso a pgAdmin son:

```
Usuario: admin@localhost.com
Clave:   admin123
```

Estas credenciales corresponden exclusivamente a pgAdmin y son independientes de las utilizadas para acceder a PostgreSQL.

Una vez dentro de pgAdmin, registrar el servidor PostgreSQL utilizando los siguientes parámetros:

```
Host:      postgres
Puerto:   5432
Base de datos: flowable
Usuario:   flowable
Clave:     flow10
```

El nombre `postgres` puede utilizarse como host debido a que los contenedores de Flowable, PostgreSQL y pgAdmin pertenecen a la misma red Docker, denominada `flowable-net`.

No es necesario publicar el puerto 5432 de PostgreSQL en el equipo anfitrión, ya que tanto Flowable como pgAdmin acceden a la base de datos mediante la red interna de Docker.

Persistencia de los datos

Los datos de PostgreSQL se almacenan en el volumen Docker `postgres_data`. Por este motivo, es posible detener y eliminar los contenedores mediante:

```
docker-compose down
```

y posteriormente recrearlos mediante:

```
docker-compose up -d
```

sin perder la información almacenada en la base de datos.

Los volúmenes existentes pueden consultarse mediante:

```
docker volume ls
```

Debe tenerse especial precaución con la siguiente instrucción:

```
docker-compose down -v
```

La opción `-v` elimina también los volúmenes asociados al proyecto y, por lo tanto, puede provocar la eliminación de la base de datos PostgreSQL.

Copia de seguridad

Puede realizarse una copia de seguridad lógica de la base de datos de Flowable mediante la herramienta `pg_dump`:

```
docker exec flowable-postgres \  
pg_dump -U flowable flowable > flowable-backup.sql
```

El archivo generado contiene una copia de la base de datos y puede utilizarse posteriormente para su restauración.

References

- [1] Business Process Model and Notation (BPMN 2.0), Object Management Group (OMG), <http://www.omg.org/spec/BPMN/2.0/>
- [2] Flowable 6.8.0 Open Source Documentation, Flowable, <https://www.flowable.com/open-source/docs/>
- [3] DevNull SMTP. <http://www.aboutmyip.com/AboutMyXApp/DevNullSmtplib.jsp>